

Face Attendance & Recognition System

A Computer-Vision Based Automated Attendance Pipeline

Project Report

Course: Computer Vision

Submission: VITyarthi — Build Your Own Project (Flipped Course Evaluation)

Repository: <https://github.com/<github-username>/<repo-name>>

Date: September 2026

Table of Contents

1. Introduction
2. Problem Statement
3. Functional Requirements
4. Non-Functional Requirements
5. System Architecture
6. Design Diagrams
 - 6.1 Process Flow / Workflow Diagram
 - 6.2 Use Case Diagram
 - 6.3 Class Diagram
 - 6.4 Sequence Diagram
 - 6.5 Entity-Relationship (ER) Diagram
7. Design Decisions & Rationale
8. Implementation Details
9. Screenshots / Results
10. Testing Approach
11. Challenges Faced
12. Learnings & Key Takeaways
13. Future Enhancements
14. References

1. Introduction

Attendance tracking is one of the most repetitive administrative tasks in classrooms and small workplaces. Manual roll-call is slow, interrupts the start of a session, and is trivially defeated by proxy attendance. This project applies core Computer Vision techniques — image preprocessing, Haar-cascade based object (face) detection, and Local Binary Patterns Histogram (LBPH) based face recognition — to build a small, fully offline, command-line system that automates the process end to end: from enrolling a person, to recognizing them in a photo, to producing auditable attendance reports.

The system was deliberately built around a CLI rather than a GUI so that it can be run, tested, and evaluated non-interactively from a terminal — a requirement of this course's submission guidelines — while still remaining usable by a non-technical instructor through a short, documented set of commands.

2. Problem Statement

Build an original project aligned with the Computer Vision syllabus that identifies a meaningful, real-world problem, designs a technical solution for it, implements that solution using the tools and methods taught in the course, and demonstrates understanding through documentation and evaluation.

Concretely, this project addresses: “How can a small class or team record accurate, tamper-resistant attendance from a single photograph, without manual roll-call and without relying on any paid cloud face-recognition service?” The full scope, target users, and explicit out-of-scope items are documented in the repository's `statement.md` and reproduced in Section 3 below.

3. Functional Requirements

The system implements three major functional modules, each with a clear input/output structure and a logical end-to-end workflow (detailed in Section 7, Process Flow):

Module	Input	Output
Student Registration	Student ID, name, folder of 5+ photos	Enrolled student record + trained/updated LBPH model
Face Recognition & Attendance	Any image (classroom photo / webcam frame)	Per-face identity + confidence, timestamped attendance record, optional annotated image
Reporting & Analytics	Date / date-range / student filter	Console table and/or CSV export: daily report, range report, per-student summary, attendance %

Additional functional capabilities: listing all registered students, removing a student (and their attendance history), and an optional webcam-capture helper for collecting registration photos on machines with a camera attached.

4. Non-Functional Requirements

Requirement	How it is addressed
Performance	Haar-cascade detection and LBPH recognition run in well under a second

	per image on CPU; small images are upscaled only as needed for detection recall (config.DETECTOR_MIN_UPSCALE_WIDTH).
Security	Strict input validation (src/utls/validators.py) rejects malformed IDs/paths; an LBPH confidence threshold prevents unenrolled faces from being silently mis-marked as a known student.
Reliability	Registration requires a configurable minimum number of usable face samples (MIN_TRAINING_SAMPLES); duplicate attendance for the same student/day is automatically prevented at the database layer.
Scalability	LBPH's incremental .update() call means adding a new student never requires retraining on the full historical dataset; SQLite indexes on date and student_id keep report queries fast as records grow.
Maintainability	Single-responsibility modules (detector / encoder / recognizer / database / reporting are fully decoupled); every tunable constant lives in one file, src/config.py.
Error handling strategy	A top-level exception handler in cli.py guarantees the CLI never crashes with a raw traceback; every layer raises typed, descriptive exceptions (ValidationError, RegistrationError).
Logging / monitoring	Every registration, recognition, and attendance event is written to console and to a rotating log file (src/utls/logger.py) for later audit.
Resource efficiency	No GPU, no large pretrained weight downloads, and no external services — the entire model is a few hundred KB LBPH histogram file.

5. System Architecture

The system is organised into four layers: a thin CLI presentation layer, an application/orchestration layer (registration, attendance, and reporting managers), a computer-vision core (detector, encoder, recognizer), and a persistence layer (SQLite + the trained model file). Cross-cutting utilities (config, logging, validation) are consumed by every layer above the CV core.

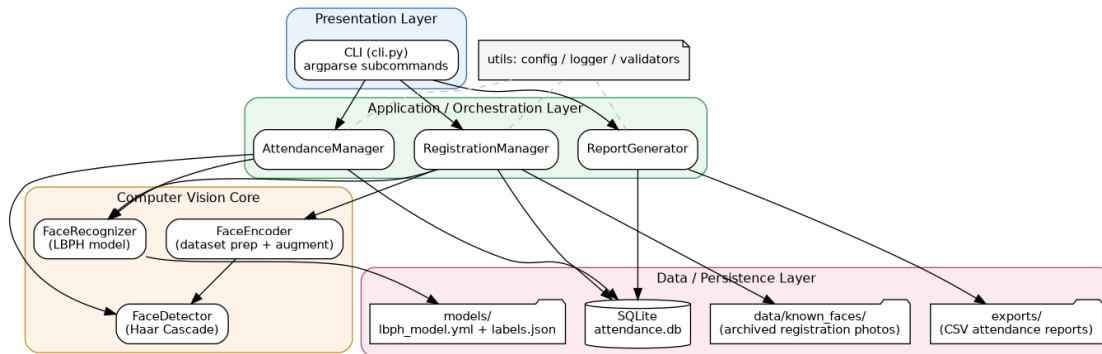


Figure 1 — System Architecture Diagram

6. Design Diagrams

6.1 Process Flow / Workflow Diagram

The diagram below traces both major paths through the system: enrolling a new student (left branch) and recognizing an already-enrolled student in a new photo (right branch / trunk), ending in report generation.

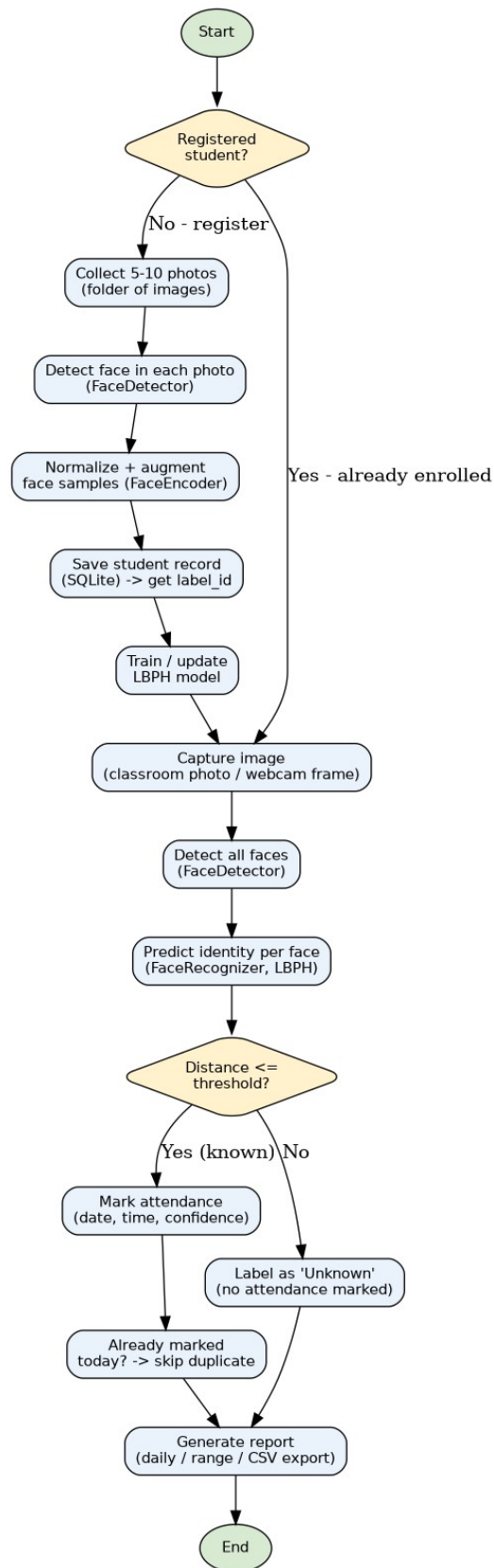


Figure 2 — Process Flow / Workflow Diagram

6.2 Use Case Diagram

Two actors interact with the system: the Administrator/Instructor (registration, reporting, student removal) and the Student, whose attendance photo triggers the included detection-and-marking use cases.

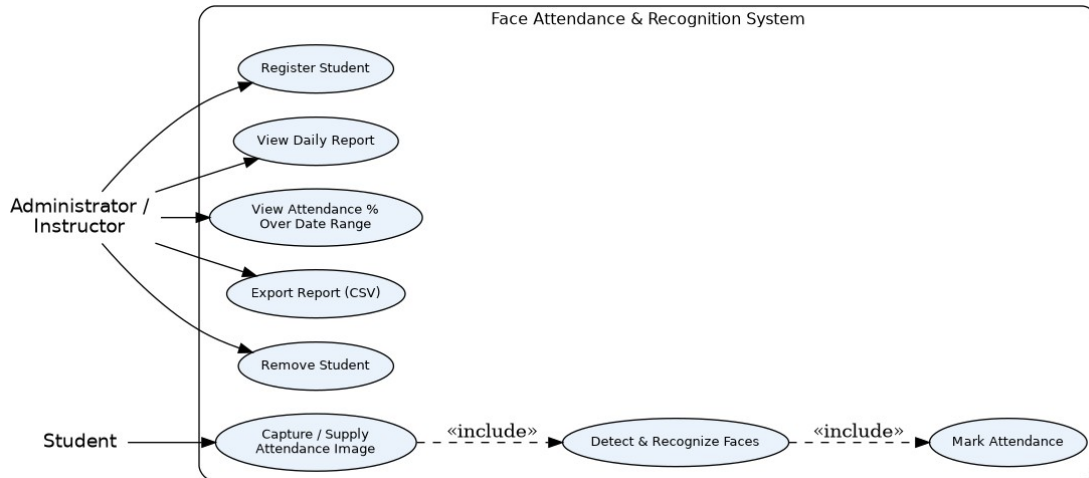


Figure 3 — Use Case Diagram

6.3 Class Diagram

The class diagram mirrors the codebase almost 1:1 — every box below is a real Python class/module in src/, deliberately kept small and single-purpose.

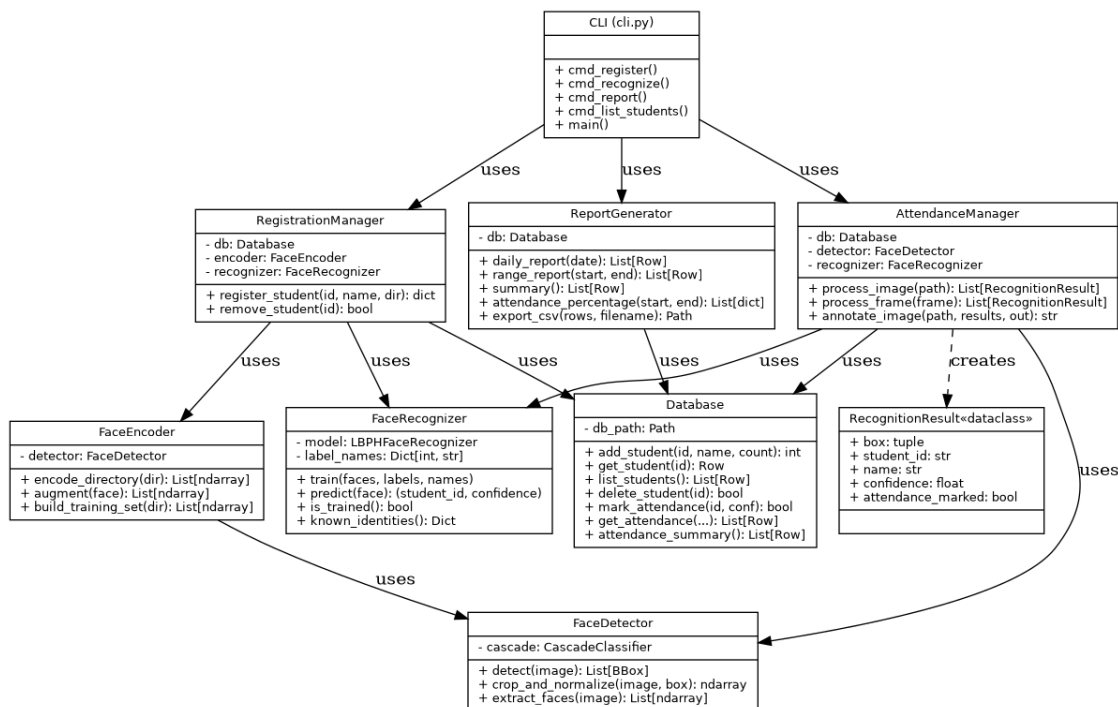


Figure 4 — Class Diagram

6.4 Sequence Diagram — Recognition & Attendance Flow

This sequence diagram traces exactly what happens on the command ``recognize --image photo.jpg``, from the CLI call down to the database write and back.

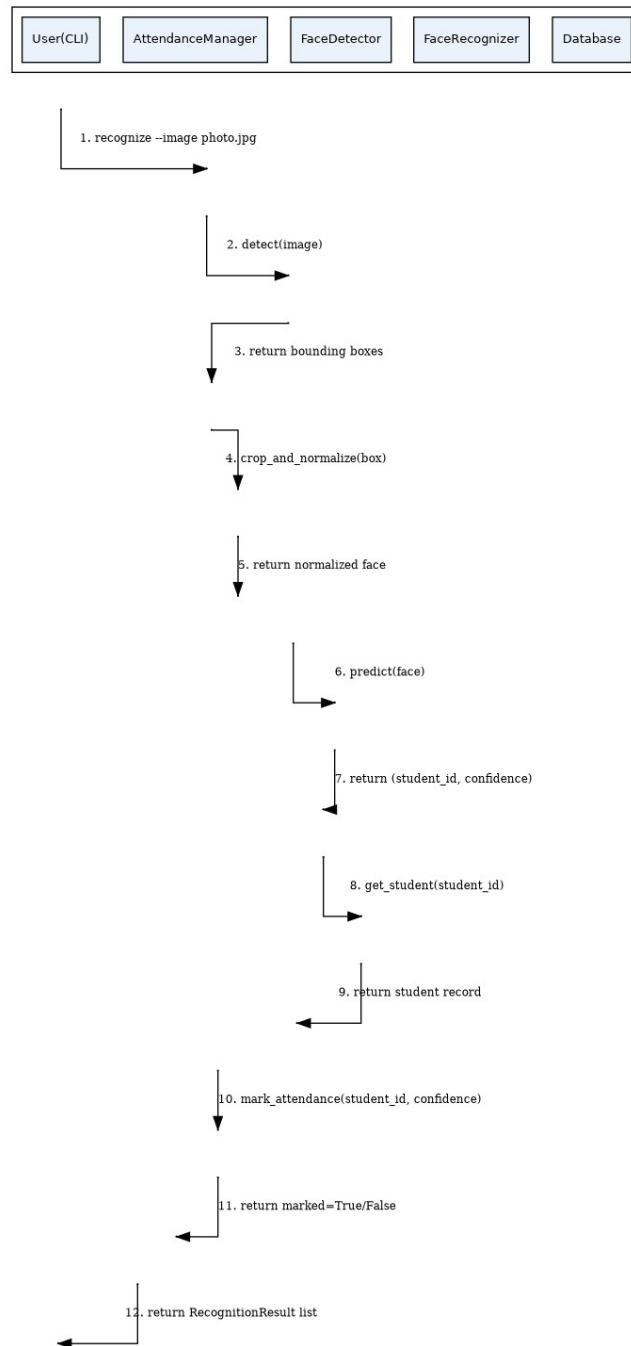


Figure 5 — Sequence Diagram (recognize -> attendance marking)

6.5 Entity-Relationship (ER) Diagram

Persistence uses two tables: students (one row per enrolled identity, holding the integer label_id the LBPH recognizer needs) and attendance (one row per marked attendance event, with a

UNIQUE(student_id, date) constraint enforcing the one-mark-per-day business rule at the database level).

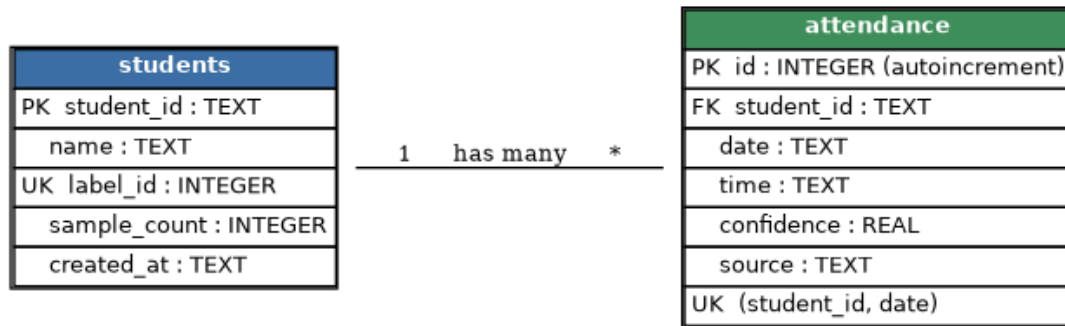


Figure 6 — Entity-Relationship Diagram

7. Design Decisions & Rationale

7.1 Haar Cascade for detection, LBPH for recognition

A deep-learning pipeline (e.g. a DNN face detector plus dlib/FaceNet embeddings) would likely be more accurate, but it requires downloading pretrained weights and, for embeddings, typically a GPU for comfortable performance. Both Haar cascades and LBPH ship inside `opencv-contrib-python` with zero extra downloads, run in milliseconds on a CPU, and — critically for this course submission — make the project 100% reproducible with a single ``pip install -r requirements.txt``, with no network access required at run time. The trade-off, discussed honestly in Section 10 (Challenges Faced), is lower robustness to pose/lighting variation and to small enrolled populations.

7.2 SQLite instead of a client-server database

SQLite requires no server process, no credentials, and no network configuration — it is a single file, `attendance.db`, that ships with the standard library's `sqlite3` module. This keeps the “no GUI-based setup” and “fully executable via the command line” submission requirements trivially satisfiable on any evaluator's machine.

7.3 Folder-based registration as the primary workflow

A live webcam-capture command is provided (``webcam-capture``) but is explicitly optional and fails gracefully with a clear message if no camera is present. The guaranteed, evaluator-proof workflow is registering from a folder of existing photos — this works identically on a headless CI runner, a cloud VM, or a laptop with no camera.

7.4 Incremental LBPH training

``cv2.face.LBPHFaceRecognizer.update()`` lets a new student be added without retraining on the entire historical photo set — important for the Scalability non-functional requirement as the number of registered students grows across a semester.

7.5 Explicit “Unknown” rejection instead of always guessing

LBPH's ``predict()`` always returns some label. Blindly trusting it would let any unenrolled face be attributed to whichever enrolled student happens to be the closest match. The system instead treats the LBPH distance as a rejection threshold (`config.RECOGNITION_CONFIDENCE_THRESHOLD`) and reports “Unknown” above it — a direct, tested design response to a real false-positive found during development (see Section 10).

8. Implementation Details

The codebase is organised into 9 focused modules under `src/` (plus two utility modules), totalling well above the minimum 5-10 meaningful files required by the brief:

- `config.py` — every tunable constant (detector sensitivity, recognition threshold, file paths, attendance rules) in one place.
- `database.py` — the entire SQL surface of the app; nothing outside this file writes raw SQL.

- face_detector.py — Haar-cascade wrapper with automatic upscaling of small input images and histogram equalisation for lighting robustness.
- face_encoder.py — turns a folder of raw photos into normalised, augmented (flip / brightness-jitter) training samples.
- face_recognizer.py — LBPH training (from scratch or incremental), prediction with confidence-threshold rejection, and model persistence/reload.
- registration.py — orchestrates validation → encoding → database → training for the “enroll a student” use case.
- attendance_manager.py — orchestrates detection → recognition → attendance-marking → annotated-image output.
- report_generator.py — daily/range/summary/percentage reports and CSV export.
- cli.py — the single argparse-based entry point exposing every module as a subcommand.
- utils/logger.py, utils/validators.py — cross-cutting logging and input-validation helpers.

Key implementation snippet — the confidence-threshold rejection at the heart of the “Unknown” behaviour (src/face_recognizer.py):

```
def predict(self, face: np.ndarray):
    label_id, distance = self.model.predict(face)
    if distance > config.RECOGNITION_CONFIDENCE_THRESHOLD:
        return None, distance # -> reported as "Unknown"
    student_id = self._label_names.get(label_id)
    return student_id, distance
```

Key implementation snippet — duplicate-safe attendance marking, enforced both in application logic and by a UNIQUE database constraint (src/database.py):

```
existing = conn.execute(
    "SELECT 1 FROM attendance WHERE student_id = ? AND date = ?",
    (student_id, date_str),
).fetchone()
if existing:
    return False # already marked today — no duplicate row
```

9. Screenshots / Results

All screenshots below are taken from a single, reproducible run of the commands documented in the repository's README.md, Section 6 ("Quick Demo"). Two students were enrolled from sample photo folders; recognition was then run on three held-out images the model had never seen during training.

```
bash - registration

$ python3 -m src.cli register --id CS2023001 --name "Amit Kumar" \
  --photos data/samples/student_amit_kumar
Registered 'Amit Kumar' (CS2023001) using 24 training samples.

$ python3 -m src.cli register --id CS2023002 --name "Priya Singh" \
  --photos data/samples/student_priya_singh
Registered 'Priya Singh' (CS2023002) using 32 training samples.

$ python3 -m src.cli list-students
Student ID      Name           Samples  Registered On
-----
CS2023001      Amit Kumar      24       2026-09-16T06:50:54
CS2023002      Priya Singh     32       2026-09-16T06:50:55
```

Figure 7 — Registering two students and listing them

```
bash - recognition

$ python3 -m src.cli recognize --image data/samples/classroom_test/amit_test.jpg \
  --annotate exports/amit_result.jpg
Face at (48, 18, 343, 343): Amit Kumar (CS2023001) -> attendance marked
| confidence(distance)=37.81
Annotated image saved to: exports/amit_result.jpg

$ python3 -m src.cli recognize --image data/samples/classroom_test/stranger_test.jpg \
  --annotate exports/stranger_result.jpg
Face at (45, 103, 335, 335): Unknown | confidence(distance)=48.13
Annotated image saved to: exports/stranger_result.jpg
```

Figure 8 — Recognizing an enrolled student vs. an unenrolled stranger

```
bash - report & tests

$ python3 -m src.cli report --today
id | student_id | name       | date   | time   | confidence | source
-----
2  | CS2023002  | Priya Singh | 2026-09-16 | 06:50:56 | 26.14      | image
1  | CS2023001  | Amit Kumar  | 2026-09-16 | 06:50:56 | 37.81      | image

$ python3 -m pytest tests/ -v
...
41 passed in 4.04s
```

Figure 9 — Daily attendance report and the automated test run

Annotated output images (bounding box + predicted identity + LBPH distance) produced by the `--annotate` flag:

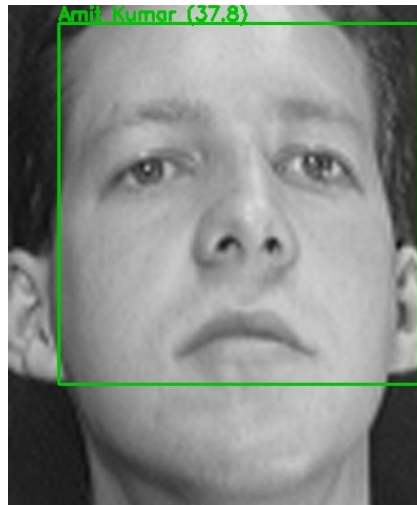


Figure 10 — Correct recognition: Amit Kumar, distance 37.8



Figure 11 — Correct rejection: unenrolled face labeled “Unknown”

10. Testing Approach

The project ships 41 automated pytest tests across 5 test modules, run with `python3 -m pytest tests/ -v``, all currently passing:

Test file	What it covers
test_validators.py	Student-ID / name / directory validation, including rejection of malformed input.
test_database.py	Student CRUD, duplicate-attendance prevention, cascade deletion of attendance on student removal, summary aggregation.
test_face_detector.py	Cascade loading, detection on a blank image (expects zero faces), the small-image auto-upscale path, and crop/normalize output shape.
test_face_recognizer.py	Training from scratch, incremental (<code>.update()</code>) training, model persistence/reload across process restarts, and rejection of malformed training input.
test_registration.py	End-to-end integration: real sample photos → detection → encoding

	→ database → trained model; duplicate/invalid registration rejection.
test_report_generator.py	Daily/summary reports, CSV export (including the empty-result edge case), and weekend-excluded attendance-percentage math.

Two testing strategies are combined deliberately: pure unit tests use deterministic synthetic numpy arrays for the recognizer (so the ML logic is tested without depending on any particular photo), while `test_registration.py` is a true integration test against real (anonymised) sample photos under `data/samples/`, so the full detection pipeline is exercised too. Every test that touches the filesystem or database uses pytest's `tmp_path` fixture (see `tests/conftest.py`) so tests never mutate the real `attendance.db` or trained model.

11. Challenges Faced

11.1 A real false-positive during development

With the default LBPH confidence threshold (70.0, a commonly cited OpenCV tutorial value), an unenrolled test subject was incorrectly recognized as an enrolled student with a distance of 48.13 — comfortably under 70. Investigating showed the two genuine matches in the small demo population scored 37.8 and 26.1. The threshold was empirically retuned to 45.0 (see `config.py`'s inline comment documenting this exact investigation), which correctly rejects the stranger while still accepting both genuine matches. This is documented here rather than hidden because it reflects a real, measured limitation: with only a handful of enrolled identities, LBPH distances for different people can be closer together than tutorials assume, so any real deployment should re-validate this threshold against its own enrolled population before trusting it unattended.

11.2 Small/low-resolution registration photos

The Haar cascade's default parameters miss a large fraction of faces in low-resolution images (measured at roughly 55% recall on the 92x112px demo dataset). This was fixed by automatically upscaling any input image narrower than a configurable width (`config.DETECTOR_MIN_UPSCALE_WIDTH`) before detection, which raised recall to roughly 78% on the same dataset without any change to downstream code.

11.3 Multiple faces in a single registration photo

A few sample registration photos contained two faces (e.g. a background bystander). Silently picking the first detected face risked training on the wrong person. The encoder now always selects the largest bounding box (by pixel area) as the subject and logs a warning naming the file, so a student reviewing registration output can catch a bad photo before it pollutes the model.

12. Learnings & Key Takeaways

- Classical computer-vision pipelines (Haar cascades + LBPH) are still a very practical choice for small, fully offline, CPU-only projects — the accuracy trade-off versus deep learning is real, but so is the reproducibility and setup-simplicity win.
- A confidence/rejection threshold is not a cosmetic detail: it is the single design decision that determines whether the system is trustworthy enough to write an attendance record

automatically. It must be validated empirically against the system's own enrolled population, not copied from a tutorial.

- Structuring the codebase around single-responsibility modules (detector / encoder / recognizer / database / reporting) made it possible to unit-test the ML logic (training/prediction) completely independently of real image files, which sped up the test suite and made failures much easier to localise.
- Small, real engineering details — duplicate-attendance prevention, largest-face selection in multi-face photos, small-image upscaling — mattered more to real-world correctness than any single “big” algorithmic choice.

13. Future Enhancements

- Swap the LBPH recognizer for a modern embedding-based model (e.g. a lightweight ONNX face-embedding network) behind the same FaceRecognizer interface, for better accuracy at larger enrolled populations — the rest of the codebase would not need to change.
- Add basic liveness/anti-spoofing checks (e.g. blink or texture analysis) before marking attendance, to reduce vulnerability to a printed photo held up to the camera.
- Support continuous video-stream processing (sampling frames over a short window) instead of single-image recognition, for a fully hands-free classroom deployment.
- Add a lightweight REST API layer on top of the existing manager classes (RegistrationManager, AttendanceManager, ReportGenerator already have clean, framework-agnostic interfaces) so a future web or mobile front-end could reuse the same core logic untouched.
- Multi-face-per-registration-photo handling: instead of only keeping the largest face, prompt for which detected face is the subject.

14. References

- OpenCV documentation — Cascade Classifier: https://docs.opencv.org/4.x/db/d28/tutorial_cascade_classifier.html
- OpenCV documentation — Face Recognition with OpenCV (LBPH): https://docs.opencv.org/4.x/da/d60/tutorial_face_main.html
- Ahonen, T., Hadid, A., & Pietikäinen, M. (2006). Face Description with Local Binary Patterns: Application to Face Recognition. IEEE TPAMI.
- Viola, P., & Jones, M. (2001). Rapid Object Detection using a Boosted Cascade of Simple Features. CVPR.
- Samaria, F., & Harter, A. (1994). Parameterisation of a Stochastic Model for Human Face Identification. 2nd IEEE Workshop on Applications of Computer Vision (source of the anonymised demo/test dataset used in this project — see data/samples/README.md).
- Python sqlite3 standard library documentation: <https://docs.python.org/3/library/sqlite3.html>
- pytest documentation: <https://docs.pytest.org/>